Gestión de uniformes por grupo

Ficha inicial de proyecto personal para registro académico

Modalidad individual | Infraestructura de Servicios | 19 de octubre de 2026

Problema y propuesta

Los pedidos de uniformes para equipos y grupos reúnen cantidades por talla, nombres, números y diseños que deben aprobarse antes de producir. Llevarlos mediante mensajes y hojas separadas puede ocasionar tallas equivocadas, versiones de diseño contradictorias y entregas incompletas. Propongo adaptar el sistema de ropa existente para controlar un pedido colectivo desde su captura hasta la entrega, con validaciones y seguimiento por etapa.

Usuarios y permisos

Representante de grupo: registra pedidos, captura tallas y personalizaciones, aprueba el diseño y consulta únicamente sus pedidos. Administrador de producción: administra prendas e inventario, revisa solicitudes, libera la producción y registra entregas. Los permisos se validarán en el servidor; aprobar un diseño corresponde al representante propietario del pedido.

Alcance funcional

Se incluirán autenticación, CRUD de grupos, prendas y pedidos relacionados, detalle por talla, personalizaciones por integrante, aprobación de diseño, reserva de existencias, seguimiento y entrega. Habrá búsqueda, paginación y un reporte de unidades pendientes por talla y etapa. Las bajas serán lógicas para preservar pedidos e historial. La semilla reproducible contendrá al menos 100 registros de negocio relacionados y cuentas ficticias.

Quedan fuera los pagos en línea, facturación fiscal, envíos externos y conexión con maquinaria. El entorno productivo será una simulación académica sin datos personales reales.

Reglas de negocio propias

1. Pedido colectivo completo. Un pedido tendrá al menos 10 prendas. La suma de cantidades por talla deberá coincidir con el total de unidades; cada unidad llevará nombre y número válidos cuando el diseño los requiera.

2. Aprobación vinculada a una versión. Solo el representante del grupo podrá aprobar la versión vigente del diseño. Cambiar diseño o personalizaciones antes de producir invalidará la aprobación y devolverá el pedido a revisión.

3. Liberación con existencias. Solo se iniciará producción con diseño aprobado y disponibilidad suficiente por prenda y talla. La reserva será transaccional y bloqueará las existencias necesarias para evitar comprometerlas dos veces.

4. Entrega completa y cancelación controlada. Solo se entregará un pedido con todas sus unidades terminadas. Se permitirá cancelar antes de producción y se liberarán las reservas; iniciado el trabajo, las correcciones se registrarán como incidencias.

Diferenciación del proyecto

La adaptación añade un agregado de pedido colectivo, aprobación versionada, reserva transaccional y trazabilidad de producción. Estas funciones modifican el proceso y el modelo de datos de la tienda; la diferenciación no depende del nombre, los colores ni del catálogo. El registro y la revisión de posibles duplicados corresponderán al docente.

Flujo y diseño técnico propuesto

Flujo con cambios de estado

Borrador → En revisión → Aprobado → En producción → Listo para entrega → Entregado.

El representante envía un borrador completo a revisión y aprueba la versión revisada del diseño. El administrador inicia producción después de reservar inventario, marca listo al terminar todas las unidades y confirma la entrega. Una corrección previa a producción devuelve el pedido a revisión; la cancelación solo procede desde borrador, revisión o aprobado. La API rechazará saltos de etapa y registrará actor, fecha, motivo y versión del pedido.

Módulos y adaptación del sistema actual

Se reutilizarán como base los módulos PHP de autenticación, usuarios, prendas, tallas, carrito y pedidos. Se incorporarán grupos y representantes, personalización, versiones y aprobación de diseño, reservas, producción e historial consultable. El carrito pasará a capturar un pedido colectivo. Los reportes mostrarán carga de producción y pendientes por talla. Estas capacidades se proponen para desarrollar; todavía no se consideran implementadas.

Tecnologías y servicios

Interfaz web con HTML, CSS y JavaScript, servida por Nginx y comunicada por HTTPS con una API PHP. El backend conservará la estructura modular de CodeIgniter y se actualizará desde la versión 3.0.4 encontrada, verificando compatibilidad con una versión mantenida de PHP. Se separará la interfaz de las vistas actuales para desplegar frontend y backend de forma independiente. Las sesiones se compartirán en PostgreSQL para permitir varias réplicas sin depender del disco local.

Docker empaquetará los servicios; Kubernetes administrará frontend, API, proceso de sincronización y ambos motores de datos. Se crearán dos clústeres independientes: desarrollo con al menos un nodo y producción de laboratorio con un control plane y dos workers. Producción tendrá al menos dos réplicas de frontend y backend y HPA de 2 a 6 en el backend. Cada ambiente tendrá HTTPS, secretos y volúmenes propios; se añadirán políticas de red, sondas, respaldos, métricas y alertas.

Uso de PostgreSQL como base relacional

Será la fuente de verdad de usuarios y roles, grupos, prendas, tallas, inventario, pedidos, detalles, reservas, estado y aprobaciones. Se usarán claves primarias y foráneas, restricciones e índices. Las transacciones protegerán reservas y cambios de estado. Las tres entidades principales con CRUD serán grupos, prendas y pedidos; los pedidos referenciarán un grupo y sus detalles una prenda y talla.

Uso y valor de MongoDB como base no relacional

Almacenará la proyección del expediente de cada pedido: versiones de diseño, personalizaciones, observaciones e incidencias e historial de eventos. Los documentos admiten atributos variables, como posiciones de bordado, colores o motivos de corrección, sin crear una columna para cada variante. El valor será visible en una pantalla de historial con filtros por pedido, etapa y fecha; no se utilizará únicamente como caché.

Se validarán los campos obligatorios y la estructura de cada evento. Habrá índices por pedido y fecha y un identificador de evento único. Los archivos de diseño se conservarán en almacenamiento persistente y el expediente guardará sus referencias y metadatos.

Consistencia y recuperación entre motores

Cada cambio y su evento pendiente se guardarán en una misma transacción de PostgreSQL mediante una tabla outbox. Un proceso enviará los eventos a MongoDB con escritura idempotente por identificador único. Si MongoDB falla, el cambio principal quedará confirmado y el historial indicará sincronización pendiente; el proceso reintentará sin duplicar eventos. Las aprobaciones y transiciones se validarán siempre contra PostgreSQL. Se probará la recuperación al restablecer MongoDB.